COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Jul 16, 2026	**DATE SUBMITTED:**	Jul 16, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 2 **Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

Screenshot

```cpp
#include <iostream>
#include <string>

class Student{
  private:
    std::string studentName;
    int studentAge;

  public:
    //constructor
    Student(std::string newName = "JohnDoe", int newAge = 18){
      studentName = newName;
      studentAge = newAge;
      std::cout<<"The constructor was called."<<std::endl;
    };
    //destructor
    ~Student(){
      std::cout<<"The destructor was called."<<std::endl;
    }

    //copy constructor
    Student (const Student &copyStudent){
      std::cout<<"The copy constructor was called."<<std::endl;
      studentName = copyStudent.studentName;
      studentAge = copyStudent.studentAge;
    }

    //display attributes
    void printDetails(){
      std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
    }

};

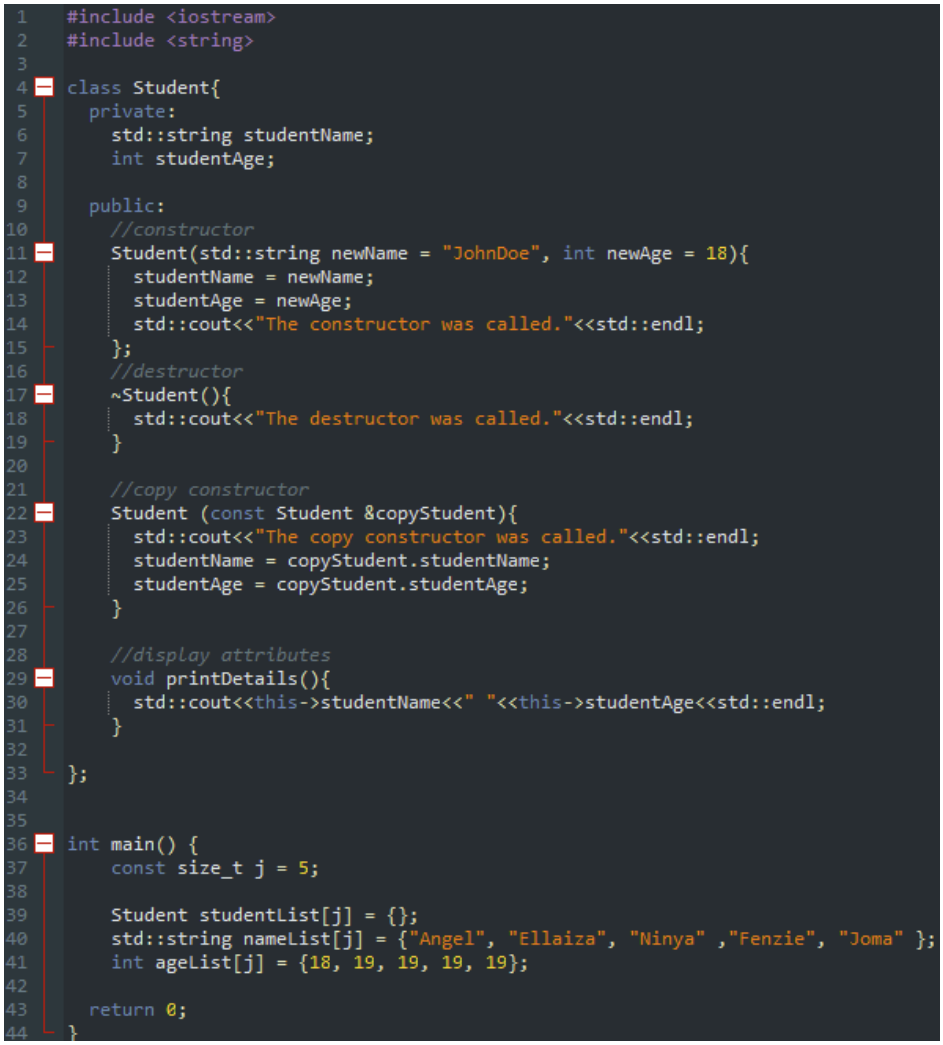

int main() {
  Student student1("Ellaiza", 19);
  Student student2(student1);
  Student student3;
  student3 = student2;

  return 0;
}
```

Observation	In the initial driver program, we can see that in line 37, a student object is being made with studentName as “Ellaiza” and studentAge “19”. The student object is created using the constructor. Once the student object is made, it then prints “The constructor was called.” verifying that line 37 was successful. The next line creates another student object but this time it is using the copy constructor. This copy constructor was called using the parameters of student1. This means that student2 will copy the attributes student1 possesses. It also prints a message verifying that the copy constructor has been called. The next student object is initialized using the default parameters of the constructor and will print a message saying that the constructor has been called, but is later assigned the parameters of student2. The parameters of student2 will replace the default parameters student 3 has been initialized with. After all the student objects have been created, it will all be deleted using the destructor as the program ends.

Table 2-1. Initial Driver Program

Screenshot



```
#include <iostream>
#include <string>

class Student{
  private:
    std::string studentName;
    int studentAge;

  public:
    //constructor
    Student(std::string newName = "JohnDoe", int newAge = 18){
      studentName = newName;
      studentAge = newAge;
      std::cout<<"The constructor was called."<<std::endl;
    };
    //destructor
    ~Student(){
      std::cout<<"The destructor was called."<<std::endl;
    }

    //copy constructor
    Student (const Student &copyStudent){
      std::cout<<"The copy constructor was called."<<std::endl;
      studentName = copyStudent.studentName;
      studentAge = copyStudent.studentAge;
    }

    //display attributes
    void printDetails(){
      std::cout<<this->studentName<<" "<<this->studentAge<<std::endl;
    }

};

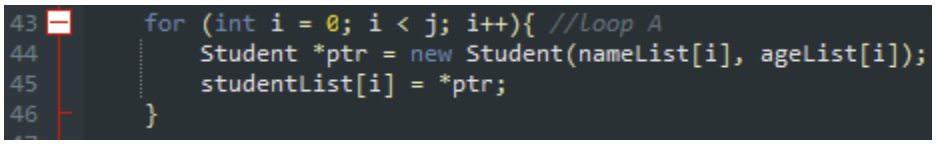
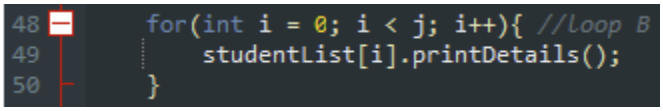

int main() {
    const size_t j = 5;

    Student studentList[j] = {};
    std::string nameList[j] = {"Angel", "Ellaiza", "Ninya" ,"Fenzie", "Joma" };
    int ageList[j] = {18, 19, 19, 19, 19};

  return 0;
}
```

Observation	The driver program starts by declaring size_t j = 5 constant. This will make it immutable meaning you aren't able to modify it after it initializes. Using size_t also guarantees that it's large enough to hold the maximum number of objects the array can handle. Then the next line of code initializes an array of student objects using default parameters of the constructor. This will initialize each item the number that the size of the array is declared as, it is called aggregate initialization. It will print the message that "the constructor has been called." 5 times as it is the number of items the array was declared to be. The next syntax will create a name list array, although this won't replace the student's name as the student object array that was created. This will be the same for age list, while it is initialized it has not really replaced the studentAge in the object array.

Table 2-1. Modified Driver Program with Student Lists

Loop A	
Observation	For this loop, I was confused with the new because it means we are creating another set of array after already creating one. We could initialize it better by directly assigning it to the student list array but I remembered that this was heap memory. This means that when using new, after using the data we should release the memory as it has done its job. So the ptr is assigned a new student array with the nameList and ageList. After the address that the pointer was assigned to is now assigned to the studentList. This allows the heap memory of the array we created to copy onto the initialized studentList array.
Loop B	
Observation	This loop will display the studentList using the printDetails function.

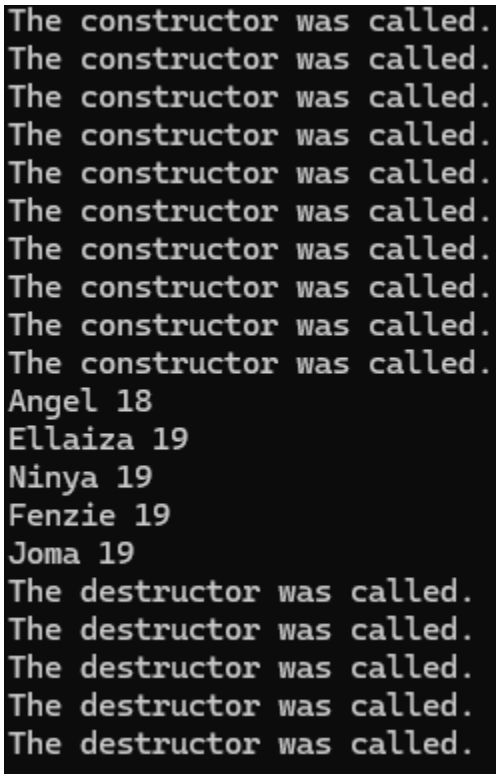
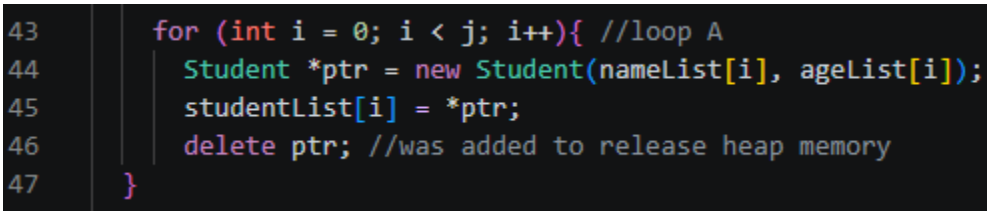
Output	
Observation	The first 5 messages of the constructor was called were the initializations of the studentList array. The 2nd set of 5 messages of the constructor was called were the initialization of the new Student array that had the nameList and ageList array. Then after, Loop B is next, which will print the details of the studentList. Then the destructor will be called to release the memory that was stored.

Table 2-3. FInal Driver Program

Modifications	
Observation	After observing the loop A before, I decided to a line of code that deletes the ptr after it was used. This is to ensure that you release the heap memory as it is no longer needed. Not releasing the heap objects lead to memory leaks. Having memory leaks causes reduced performance, resource depletion, and the possibility of your program crashing.

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

N/A

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I had it explain the topic so that I could better understand the code and concept. AI was used and it did not replace original thinking, analysis, authorship, or completion of the required work.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.